READ THIS OUT LOUD. DO NOT SKIM THE SHORT LINES.

How to think in this interview

This is for Meshack. Tomorrow is Influx, Technical Support Agent L2. The last version of this pack was too tight. This one is written the way you should speak: full sentences, one thought after another.

What they actually want

They want a person who can sit at home in Kenya, take a difficult ticket, and not make the system worse. The product will be new to you. That is fine. They train you on their buttons. What they cannot train in a week is judgement.

So the job looks like this. A customer has a problem that the first person could not finish. You open the ticket. You try to see the same problem yourself. You look at the live picture of the system. You read the logs. If the written guide says you may restart something or merge an already approved change, you do that. If the next step would change live customer data and you cannot put it back, you stop. You write down what you saw. You send that to Engineering. You tell the customer what is going on, in ordinary English. You can do that at night as well as in the day.

What L1 does, and what L2 does

Think of a hospital help desk, because that is the world you already know.

The L1 person is the front desk. Someone says the system is not working. L1 checks the simple things. Are you logged in? Did you try again? Is your password expired? They follow a short article. They are polite. They write a clean note. When the article is finished and the thing is still broken, they do not start guessing. They send the ticket to L2.

The L2 person is you. You are not guessing either. You are going deeper. You try to make the problem happen again. You look at logs. You look at whether something was changed this afternoon. You ask yourself which layer is broken. Is it the person's account? Is it the network? Is it the application? Is it the data? Then you do only the fix that is already allowed. If you cannot finish it safely, you prepare a package for the people who write the product. That package is not "it does not work." It is "here is how I made it fail, here is the time, here is the log, here is what I already tried."

Engineering changes the product. You do not pretend to be Engineering. You make their night shorter.

This is already your day at PATH

Do not walk in tomorrow feeling like a stranger. You have been doing this loop on health systems. Influx has a different product and different tools. The thinking is the same.

When a facility says KenyaEMR is down, you do not immediately restart a server. You check whether one person cannot log in, or whether the whole site is stuck. You look at Grafana or the live picture. You look at identity. You look at the network. You look at whether something changed. You read logs. You write Python or SQL when you need to see the lie in the data. You have Freshdesk notes that another person can pick up. You have trained people in ordinary English. You do not run a write on a live database because someone is shouting.

GitHub is the console you already use. GitLab is the same idea on another website: make a small change, ask someone to look, merge it, and if it is wrong you reverse that change. Kubernetes is how they run their application in pieces. You are not the owner of that cluster. You are the person who can find the sick piece, read why it died, and restart it if the guide says so.

The thinking process they are scoring

On almost every technical question, walk the same path in your head. Say the path out loud. They mark the path more than they mark a perfect tool name.

First: what is actually happening? One user or many users. Can I see it myself.

Second: is the system really sick, or does one person just feel that it is? That is why you look at Grafana or a health page before you restart anything.

Third: which layer. Account. Network. Application. Data. Do not skip ahead.

Fourth: what changed recently. If something was released twenty minutes ago and the site died, that release is guilty until the logs say otherwise.

Fifth: is there an allowed fix? Restart. Merge the waiting change. Add space. If yes, do it and write it down.

Sixth: would the next click change live data in a way I cannot undo? If yes, stop. Write the evidence. Call the owner of that data.

Seventh: tell the customer what you know, what you are doing, and when you will speak again. Do not invent a time you do not own.

If you go quiet for three minutes while you think, they will think you are lost. Talk through the steps. "I am checking whether this is one user or everyone. Then I will look at the live picture. Then the logs." That is what good sounds like.

THE TWO RULES AND THE TOOLS, IN ENGLISH

What you must be able to show

You do not need to memorise a dictionary. You need two rules, a few words, and four small pieces of work you can write if they hand you a keyboard.

Rule one. Some actions you can undo. Some you cannot.

You can undo a restart. The service comes back, or you start it again. You can undo a merge request if you reverse that same merge. You can undo a config change if the old version is still there.

You cannot undo a delete on a live database. You cannot undo dropping a table. You cannot undo a bulk update that overwrote real customer rows. You cannot undo wiping a disk to free space. If you cannot put it back in one clean step, you do not do it, even if the ticket is a P1 and the customer is angry.

This is the question they will keep asking until they hear you stop. If you only remember one answer tomorrow, remember this one.

Rule two. A workaround is not the same as a fix.

An incident is the fire in front of you. The reporting page is failing. Your job is to get people working again.

A change is the allowed work you do on purpose. You restart the worker. You merge the request that was already approved.

A problem is the reason the fire keeps coming back. A bad mapping that will lie again tomorrow night. You can put a workaround on the incident so the queue survives. You do not close the deeper problem and walk away as if you healed the product. Engineering owns that part.

Kubernetes in ordinary words

Imagine the application is a shop with several cashiers. A pod is one cashier at a till. A deployment is the manager saying "I want three cashiers. If one walks out, replace them." A service is the single door customers walk through, so they do not care which cashier they get.

CrashLoopBackOff means a cashier keeps sitting down, standing up, and falling over again. Restarting them over and over does not help. You ask why they fell. You read the last note they left, which in this world is the previous log. If they ran out of memory the note often looks like exit code 137. If the program itself crashed, it often looks like exit code 1.

You type these in this order, and you say why as you type. First you list what is running. Then you read the story of the sick one. Then you read its logs. Then you read the logs from the last time it died, because the new copy is often empty.

```
kubectl get pods -n prod
kubectl describe pod PODNAME -n prod
kubectl logs PODNAME -n prod
kubectl logs PODNAME -n prod --previous
```

If the guide says you may restart that group of cashiers, you restart the deployment. If a release twenty minutes ago caused the fall, and the guide allows a rollback, you roll back first and investigate after people can work again.

```
kubectl rollout restart deployment/NAME -n prod
kubectl rollout undo deployment/NAME -n prod
```

You do not paste random files into the cluster. You do not climb inside a pod to "just fix production." You do not delete a disk to make space unless that exact sentence is in the guide.

GitHub and GitLab in ordinary words

You do not change the live shop by hand. You write the change in a small branch named after the ticket. Someone looks at it. Then you merge it. If it is bad, you reverse that merge. That is a merge request. GitHub is where you already do this. GitLab is the same walk on another site. Say that once. Do not give a lecture.

```
git checkout -b fix/ticket-1842
git add .
git commit -m "ticket-1842: apply the approved change"
git push -u origin fix/ticket-1842
```

How to write a Python script if they ask

They will not ask you to invent a clever algorithm. They will ask you to prove you can read a log. The recipe is always the same. Open the file. Look at each line. Keep the ugly lines. Print how many you found, then print the first twenty so a human can see them. If they want to know when it spiked, group the lines by the hour in the timestamp.

This is the same habit as your PATH checks. You already open a file, look for the bad rows, and count them. Tomorrow you may do it on their log instead of a reporting export.

Script you should be able to write from memory

```
# python3 find_errors.py app.log

import sys

path = sys.argv[1] if len(sys.argv) > 1 else "app.log"
hits = []

with open(path, encoding="utf-8", errors="replace") as f:
    for line in f:
        if " 500 " in line or " ERROR " in line or "Traceback" in line:
            hits.append(line.rstrip())

print("I found", len(hits), "bad lines")
for line in hits[:20]:
    print(line)
```

If they put a Python error on the screen, do not start at the top. Start at the bottom. The last line is the name of the failure. ConnectionError means it could not reach something. KeyError means a value was missing. Timeout means it waited and gave up. Then look one or two lines up and name the file. Then say the next safe action. Usually that is not "run a write."

A tiny check before you restart anything

```
# python3 health.py https://api.example.com/health

import sys
import urllib.request

url = sys.argv[1]
try:
    with urllib.request.urlopen(url, timeout=8) as r:
        print("status", r.status)
        print(r.read().decode()[:200])
except Exception as e:
    print("FAILED", type(e).__name__, e)
```

You run this to answer a simple question: is the door even answering. A page can look fine to one user and still be lying. That is your reporting story.

If they ask for SQL, keep it a read

```
SELECT DATE_FORMAT(created_at, '%Y-%m-%d %H:00') AS hour,
       COUNT(*) AS fails
FROM   requests
WHERE  status_code = 500
  AND  created_at >= NOW() - INTERVAL 24 HOUR
GROUP  BY hour
ORDER  BY hour;
```

This only reads. It counts failures by hour so you can see the spike. If they ask why a query is slow, you say you would look at the plan first before anyone talks about adding an index on production.

TWENTY QUESTIONS. THINK, THEN SPEAK.

Practice in your own voice

Read the thinking. Then say the answer out loud as if a person is sitting across from you. Do not recite keywords. Tomorrow, start with the first sentence. If they cut in, let them, then continue the path.

Question 1

What does L1 do, and what will you do here as L2?

How you think about it

They want to hear that you will not stay on the surface, and that you will not play engineer. Draw the line with a story, not with job titles.

What you say

L1 is the front desk. They confirm who the user is, they walk the written article, and when the article is finished they escalate. I am L2. I take the ticket after that. I try to see the problem myself. I read the logs. I do the fix that is already allowed. If the next step would change live data and I cannot put it back, I stop and I send Engineering a real package: the time, how I reproduced it, the log, and what I already tried. I still talk to the customer. I just will not guess.

What you can show

At PATH you are already that second person. About 120 people reach you when the first instructions fail.

Question 2

A customer says the application is down. What do you do first?

How you think about it

They are scoring the order. If you jump to a restart, you sound dangerous. If you jump to the database, you sound worse. Walk the path.

What you say

First I try to see it myself. I need to know if this is one person or everyone. Then I look at the live picture, Grafana or the health page, so I am not chasing a feeling. After that I check the account, then the network, then whether something was changed recently. Then I read the logs around that time. If the guide names a restart or an approved merge, I do that and I write it down. If the next click changes live data, I stop. In the first few minutes I also put a note on the ticket so the customer is not sitting in the dark.

What you can show

This is the same order you use when a facility says the EMR is down and it turns out to be a login, a bad link, or a reporting lie.

Question 3

What would you never do on production?

How you think about it

This is the job. Say it slowly. Do not decorate it.

What you say

I would never write to a live production database without the owner saying yes. I will not delete rows. I will not drop a table. I will not run a bulk update I invented because the ticket was loud. I will not change the shape of the database at two in the morning. A restart I can undo. A merge I can reverse. A row I overwrite is gone. "I thought it would work" does not bring it back.

What you can show

That is already how you treat KenyaEMR and MySQL. Keep the story in health systems. You do not need a bank.

Question 4

How do you decide if an action is safe?

How you think about it

Translate reversible and not reversible into a question you ask yourself.

What you say

I ask one question. If this goes wrong, can I put it back in one clean step? If yes, and the guide allows it, I do it. Restarting a service is like that. Reversing a merge is like that. If the answer is no, I do not do it. Changing live rows, wiping a disk, or editing the live system by hand is like that. Pressure does not change the answer.

What you can show

If they keep pushing, repeat the question. "Can I put it back?" That is the whole idea.

Question 5

Tell me about a time the first instructions failed and you still finished the work.

How you think about it

They want a real day, not a superhero story. Use the reporting incident. Stay in what you actually did.

What you say

We had a reporting integration that started lying. On the screen everything looked fine. The numbers on the national side did not match. People called it an outage. It was not an outage. It was a bad transform. I made it happen again on one site. I pulled the logs. I ran SQL on both sides. I compared the API payload. The browser was fine. The login was fine. The network was fine. I sent Engineering and QA the path to make it fail, the times, the request and the response with personal data removed, the SQL, and the things I had already ruled out. They got a real defect. The people in the queue got a workaround until the mapping was fixed.

What you can show

That package is exactly what this job calls an escalation. Practise it once slowly tonight.

Question 6

What do you send to Engineering so they can actually help?

How you think about it

If your escalation is "it is broken," you have not done L2 work. List the pieces as if you are leaving for the night.

What you say

I send a short picture of the damage. When it started. How to make it happen again. The request and the response, with names and numbers that can identify a person taken out. The log lines. What I already tried. What is safe to run. What I have already told the customer. If I could not reproduce it, I say that honestly. I do not send a feeling.

What you can show

Open one of your better Freshdesk notes in your head. That is the standard.

Question 7

What is an incident, what is a change, and what is a problem?

How you think about it

They use these words. Give one example each from your own work so it does not sound memorised.

What you say

An incident is the fire in front of me. The page is failing and people cannot submit. My job is to get them working. A change is the allowed work I do on purpose, like merging an approved request or restarting a worker the guide names. A problem is the reason the fire will come back, like a mapping that will lie again tomorrow. I can leave a workaround on the incident. I do not pretend the workaround healed the product.

What you can show

If they only want two sentences, use fire, allowed work, and the reason it will return.

Question 8

Three tickets all say P1. How do you choose?

How you think about it

They know first line marks everything urgent. They want to hear you look at damage, not at the stamp.

What you say

I look at who is actually stuck, not at the label. If people cannot enter clinical data or cannot complete a payment, that goes first. An internal report that can wait goes later. I still answer all three quickly so nobody feels dropped. I tell the other two when I will come back. If I need another person, I say so. I do not hide three fires in one lap.

What you can show

A clinic that cannot enter data beats a dashboard that looks ugly. You already know this.

Question 9

Something was released twenty minutes ago and now the site is failing. What do you do?

How you think about it

A lot of people try to be clever and keep digging while customers sit in the dark. The strong answer is restore first.

What you say

I treat that release as guilty until the logs say it is not. If the guide allows me to roll it back, I roll it back first and I investigate after people can work. I would rather have yesterday's working version than a clever theory on a dead site. If they keep changes in GitLab, the clean undo is to reverse that merge, not to poke the live cluster and walk away. Then I write the time we rolled back and I stay until the live picture is quiet.

What you can show

This is the same instinct as undoing a bad change instead of editing more things on top of it.

Question 10

How do you use Python at work, and when would you refuse to run a script?

How you think about it

They need to hear that Python is a lamp, not a weapon. Offer to write the small log script.

What you say

I use Python to see clearly. I open logs. I count errors. I check a health address. I validate a file so a human is not doing the same monthly grind by hand. At PATH that habit cut a reporting job by about forty percent. I will not run a script that writes to a live database. I will not run a script I have not read. If you put an error on the screen I start at the bottom, I name the failure, I name the file, and I say the next safe step.

What you can show

If they say "show me," write the find_errors script from the previous page. That is enough.

```
with open("app.log") as f:
    hits = [line for line in f if " 500 " in line or " ERROR " in line]
print(len(hits))
print("".join(hits[:20]))
```

Question 11

How do you use GitHub or GitLab, and why not change production by hand?

How you think about it

They wrote merge requests on the job post. Explain the walk. Then stop.

What you say

I make a small branch named after the ticket. I put the change there. I open a merge request. Someone looks at it. Then I merge it the way the guide says. If it is wrong I reverse that merge. That gives us a history and a way back. Changing the live system by hand has no review and no clean undo. I live in GitHub and I have the GitHub Professional certificate. GitLab is the same walk on another site. I will not spend a week lost in the menu.

What you can show

Do not say you run their GitLab. Say you already know the walk.

Question 12

Explain a pod, a deployment, and a service as if I am not a Kubernetes person.

How you think about it

Use the shop. If you start saying mesh and ingress you will sound like you memorised a poster.

What you say

A pod is one running copy of the application. Think of one cashier. A deployment is the manager saying how many cashiers we want, and replacing one if they leave. A service is the single door in front of them, so a customer does not care which cashier they get. If the door is open but people still get errors, I do not stop at "the cluster is fine." I check whether the cashiers behind the door are ready, falling over, or serving errors.

What you can show

If they go deeper than this, say you will learn their platform. Do not invent the rest.

Question 13

A pod keeps dying and the status says CrashLoopBackOff. What do you do?

How you think about it

The status is not the bug. The last log is the bug. People fail this when they only restart.

What you say

It means the copy starts, dies, and the platform keeps trying again. I do not sit there restarting it. I find that pod. I read the story of why it last died, including the exit code. I read the logs. Then I read the logs from the previous death, because the new copy is often empty. If the exit looks like memory, that is one path. If it looks like the program crashed, that is another. I also look at what was released. If the release caused it and the guide allows a rollback, I roll back. I do not climb inside the pod to look clever.

What you can show

Say the four commands in order and say why. List, story, logs, previous logs.

```
kubectl get pods -n prod
kubectl describe pod PODNAME -n prod
kubectl logs PODNAME -n prod
kubectl logs PODNAME -n prod --previous
```

Question 14

When do you restart something, and what do you do if the disk is full?

How you think about it

They put restarts, storage, and capacity in the job post. The hidden test is that you wait for the guide.

What you say

I restart only when the written guide names that service or that deployment, and only after I have looked at the logs. Restarting a config error just makes a louder loop. If they need more capacity I add copies only if the article allows it, then I watch the live picture. If a disk is full I do not delete a volume to make space. That can destroy data. I escalate if the full disk is the data disk. A restart I can undo. Wiping a disk I cannot.

What you can show

The phrase "inside documented guidelines" is the job. Repeat it in your own words.

Question 15

The board is red. The ticket only says the app is slow. Are those the same thing?

How you think about it

They want you to connect monitoring to the human, not to treat them as one blob.

What you say

Not until I prove it. The board tells me if the system is sick. The ticket tells me who is hurting. I look at the same window of time. Are errors up. Is it slow for many people. Was there a release. If the board is quiet and one person is slow, I look at their network, their browser, and their account. If the board is red and many tickets arrive, I treat it as one incident and I stop doing the same work twice.

What you can show

You already use Grafana this way. You have not owned Datadog. Do not add it.

Question 16

The customer is not technical. Tell them what is going on, then tell me what you write on the ticket.

How you think about it

This may be the English test. Speak like a person. Then write the same facts.

What you say

To the customer I would say: we have confirmed the break on our side. Your data is not gone. We have a way for you to keep working. Engineering has the defect. I will update you in thirty minutes even if nothing new has happened. On the ticket I write who is affected, when it started, what I tried, the workaround, who owns the deeper fix, and when the next update is. I will not promise a clock I do not own.

What you can show

Say the customer line out loud twice tonight. Short. Calm. No extra words.

Question 17

The target is four hours. You are at three hours fifteen and you still do not have a fix. What now?

How you think about it

They fail people who hide in the logs hoping to be a hero at hour four.

What you say

I escalate before the clock breaks, not after. I send a short brief. This is the damage. This is what we know. This is what we tried. This is what I need. We are inside the last hour. I keep the customer on a thirty minute update. I do not disappear into the logs and hope I will save it at the last minute.

What you can show

Early, written, calm. That is the whole answer.

Question 18

Your shift is ending and the incident is still open. What do you leave behind?

How you think about it

Night work dies when the outgoing person leaves a mystery. They are testing whether they can trust you at 6 a.m.

What you say

I write as if I will be asleep and a stranger has to finish. I leave the timeline. I leave what I tried. I leave the logs and the links. I leave what is safe to run. I leave what the customer already heard. I leave the next check time. I make sure the incoming person has the ticket in their hand. I do not just clock out and hope.

What you can show

If you have ever handed a Freshdesk ticket to someone on the next shift, tell that in one line after the answer.

Question 19

This job is forty four hours, including nights, weekends, and holidays, from home. Can you do that?

How you think about it

If you hedge, the interview is over. They need a body on the roster.

What you say

Yes. I can work days, nights, weekends, and holidays. I work from Kisumu. I have a quiet room, a Windows machine, a headset, and a backup way to get on the internet if the home line dies. I will send a live speed test. A ticket at two in the morning gets the same care as a ticket at two in the afternoon. I follow the guide. I read the logs. I stop before live data. I update the customer.

What you can show

Have the speed number ready. Do not give a speech about power cuts. Say you have a backup.

Question 20

You already have a job at PATH. Why this seat at Influx?

How you think about it

They want to hear you chose L2, not that you are running from PATH or chasing any remote job.

What you say

I want this kind of seat, not a chat queue. The work I already like is this work. I read the logs. I run a small script. I do the safe change. I give Engineering something they can use. PATH taught me that habit inside a careful health system. Influx is Kenya, work from home, international customers, and a team that sits next to the product. I can start when you need me. I will learn your GitLab and your Kubernetes the way I learned every other console. With the ticket, with the guide, and with the person who is waiting.

What you can show

If they ask for money, let them name their band. The form used 150,000 shillings net. Do not invent a bank salary.

IF THEY PUSH, AND HOW YOU START TOMORROW

Stay in what you can honestly show

They may try to make you bigger than you are. The strong move is a full sentence that tells the truth and still sounds useful.

If they keep asking about Kubernetes

I use Kubernetes for careful work. I find the sick copy. I read why it died. I restart or I roll back if the guide says I may. I am not the owner of your cluster. I get used to a new console the way I got used to every other one. With the ticket and with the guide. I will not climb into production to look sharp.

If they keep asking about GitLab

GitHub is the place I already work. I have the GitHub Professional certificate. GitLab is the same walk: branch, request, review, merge, reverse if needed. I will not lose the first week on the menu.

If they mention computer vision, Datadog, banks, or airports

I have not supported a computer vision product. I have not owned Datadog. I use Grafana in real work. My careful environment is health information systems. We control access. We do not make wild changes. We take people's details out of screenshots. That is the same muscle, in a different building.

If they put a Python error on the screen

Start at the bottom. Name the failure in English. Name the file. Then say the next safe step. For example: this is a ConnectionError to the database in sync.py. The website may still be up. The thing it needs is not. I would not restart the website in a loop. I would check whether the database is healthy, whether something just changed, and I would call Infrastructure if the database is the break.

If the customer asks you to just fix the database

I can read it. I can show Engineering the bad rows. I will not write to production without the person who owns that data. That is how we keep what you already entered.

If they want a time you do not have

I will not give you a time I cannot keep. We have confirmed the break. The workaround is in place. I will speak to you in thirty minutes. If we know a real time before that, I will call you sooner.

Tomorrow morning

Read this first page again. Then say question 3, question 5, question 13, question 16, and question 19 out loud, sitting up, as if a person is there. Those five are the ones that end the interview if they go badly. Never write the database. The real PATH story. The dying pod. The customer English. The night.

Have water, a headset, a charger, and a quiet room. Have a speed number. Mail from Influx ends with influx.com. You do not pay anyone for this job.

If they share a screen, talk. Say I am checking whether this is one user or everyone. Then I will look at the live picture. Then the logs. Then I will tell you if a restart is allowed. If you do not know a product name, say so and keep walking the path. A true path beats a fake tool.

You can do this work. Tomorrow they need to hear you think, not hear you stack short words. Follow the guide. Read the logs. Stop before live data. Tell the person waiting. Say yes to the night.